7 A QUANTUM ONTOLOGY

In this chapter, we present a complete example of rewriting, starting from an ontology expressed in OWL (Section 4.1.3), and arriving at a Prolog formulation suitable for the pipeline presented in the previous chapter.

Following the literature, we begin with the translation of the ontology into Prolog, discussing how to translate concepts such as classes, subclasses, individuals, and relations. We then focus on the incompatibility of this first version with QA-Prolog, highlight the reasons for this incompatibility, and propose possible solutions.

We present some examples of queries to demonstrate the equivalence of the two transformations. Finally we provide observations on the final result obtained after the transformations performed by QA-Prolog.

7.1 STARTING ONTOLOGY

The ontology we will translate in Prolog is not the one presented in Chapter 4 but is a more complex one in order to show an example more similar to a real ontology.

The domain of interest of our ontology is the animal kingdom; this domain serves as a pretext for a deeper description than the one shown in the family tree example (Section 4.2.1). We will see how to describe animal species, their feeding habits, and the relationships between individuals.

7.1.1 Ontology structure

In this ontology, we describe a small part of the animal kingdom. The more complex class hierarchy represents the taxonomy of animals: it characterizes living beings with increasing precision until identifying their species.

The other classes focus, instead, on feeding habits of animals. We distinguish between prey and predators, and between carnivorous and herbivorous animals (in this small example, for simplicity, we ignore omnivores[1]).

Finally, we consider relationships between animals. We define two types of relationships: parental relation that connect two animals in parent–son roles, and relation related to feeding habits that link two animals in the roles of predator and prey.

The T-Box of our ontology can be seen in Figure 7.1, we define also some individuals and their relations, to populate our ontology[2].

1 The reason will be clear in the next section.

2 The code for this ontology is available at `https://github.com/DavideCamino/Thesis/blob/main/Code/Ontology/animals.rdf`.

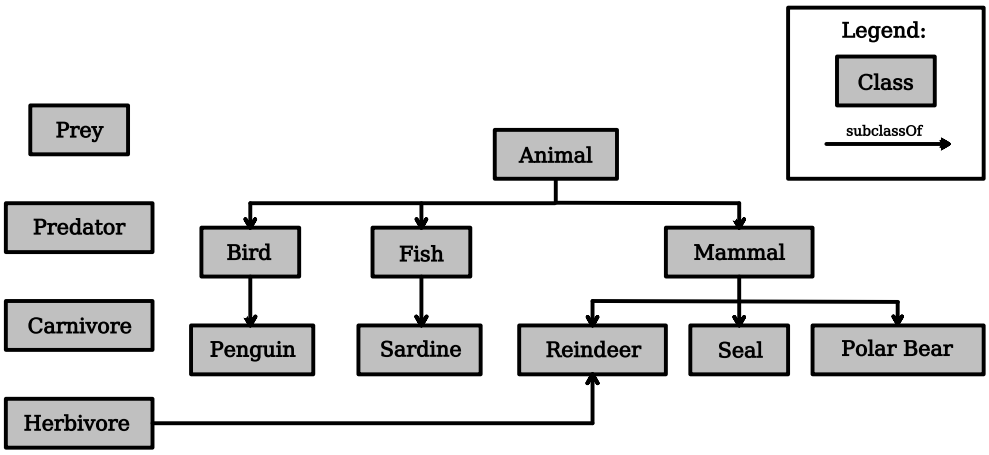

Figure 7.1: Ontology T-Box

7.1.2 Ontology constraints

In this section, we present the constraints on classes and individuals represented in our ontology. These constraints allow us to make inference on the ontology and derive new information from what is already present. Thanks to these constraints we can determine if all the information contained in the ontology is consistent with the theoretical model we are describing.

Constrains defined in animal ontology are:

- Constrain on classes:
  - carnivore and herbivore are disjoint classes;
- Constrain on individual:
  - If animal A hunts animal B, A is a predator and carnivore and B is a prey.

From defined constraints we can see that a prey and predator are not disjoint classes and being a prey does not automatically mean that the animal is a herbivore. This is coherent with the animal kingdom where a certain animal could be both a prey and a predator.

It should also be clear why we have omitted the omnivores from our description: this simplification allows us to describe herbivore and carnivore as disjoint and to infer that an animal that hunt is automatically a carnivore.

7.1.3 Ontology contents

We now examine some facts represented in the ontology; these will serve as examples to show how integrity checking of the information collected in the KB, and the reasoning processes work.

Let us begin by considering the classes. As shown in the T-Box of Figure 7.1, the animals we can describe are: sardines, penguins, seals, polar bears, and reindeers. From the T-Box we also see that we have only specified reindeer as herbivorous; feeding habits of other spices can be inferred with the hunt relationship.

We now examine some facts about the individuals that populate our KB:

- Beary is a polar bear and hunts Sealy and Penguy;
- Sealy is a seal and hunts Penguy;
- Penguy is a penguin and hunts Sardy;
- Sardy is a sardine;
- Reiny is a reindeer and hunts Sardy;
- Reiny_b and Reiny_c are reindeers, Reiny_b is son of Reiny and Reiny_c is son of Reiny_b.

From this information, we can deduce that Beary and Reiny are predators, Sealy and Penguy are both predators and prey; and Sardy is a prey.

CONTRADDICTION: A contradiction has also been deliberately introduced into the ontology. From the fact that Reiny hunts, it follows that she is a carnivorous animal; however, her species is labeled as herbivorous. Reiny is therefore both herbivorous and carnivorous. Moreover, herbivores and carnivores are disjoint classes. This makes the KB inconsistent.

If we used a reasoner to verify the consistency of the KB, for example HermiT [13], we would obtain an error message informing us that the ontology is inconsistent and that the reasoner's results cannot therefore be considered reliable.

The reasoner is also able to provide an explanation of the inconsistency (shown in Figure 7.2[3]), from which it is clear that the problem is that Reiny is simultaneously herbivorous and carnivorous.

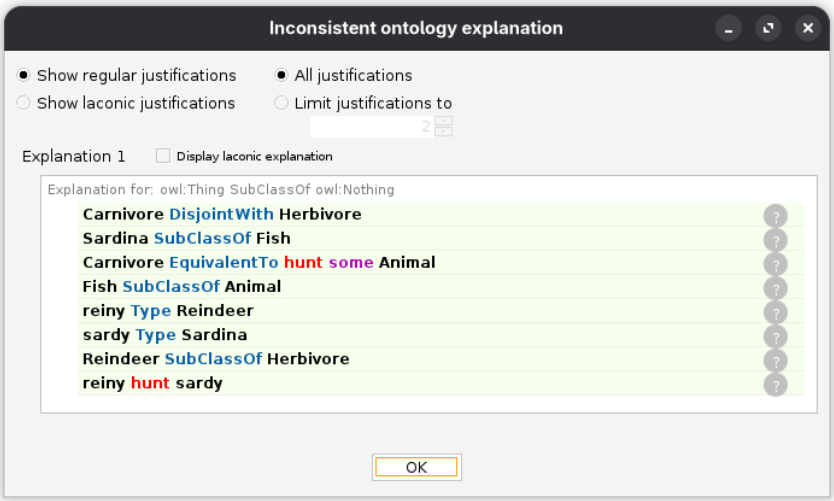

Figure 7.2: Error dialog in Protégé

We introduced this inconsistency in order to show —in the following sections— how it is also possible in Prolog to detect this type of error and identify contradictions in the KB. If we remove the fact "Reiny hunts Sardy" the ontology will return consistent.

3 The figure shows the dialog window of Protégé[36], a widely used ontology editor.

7.2 PROLOG VERSION

To translate the ontology into a Prolog KB, we follow the work presented in [37]. This article discusses in depth how to replicate the various constructs provided by OWL in order to obtain a logically equivalent KB in Prolog.

In this section, we will see how to translate the concepts of class and subclass, disjoint classes, and relationships between individuals; we will present rules to infer new information, as was done in the ontology. Lastly, we will show how to detect errors and inconsistencies in the KB.

Our goal is not to go through the entire translation of an ontology into Prolog, but rather to convince ourselves that such translation is possible.

The original ontology includes only a small subset of the constructs available in OWL. This is because the translation into Prolog is only the first step; in the following sections, we will modify the Prolog program to make it compatible with QA-Prolog and obtain a representation suitable for a quantum annealer.

Advanced topics such as the open world assumption, enumerated classes, existential quantification, cardinality rules, cyclic hierarchies, and more are covered in [37].

7.2.1 Classes and subclasses

We start our KB describing the T-Box of the ontology: here we define the classes' hierarchy. Listing 7.1 shows how to build the hierarchy: (a) is a list of facts that defines the classes (for example, line 1 can be read as "animal is a class."), (b) defines the structure of the classes (line 1 is read as "mammal is a subclass of animal.").

```
1  class(animal).
2  class(mammal).
3  class(polar_bear).
4  class(seal).
5  class(reindeer).
6  class(fish).
7  class(sardine).
8  class(bird).
9  class(penguin).
10 class(herbivore).
11 class(carnivore).
12 class(prey).
13 class(predator).
```

(a) Classes definitions

```
1 subclass(mammal, animal).
2 subclass(bird, animal).
3 subclass(fish, animal).
4 subclass(polar_bear, mammal).
5 subclass(seal, mammal).
6 subclass(reindeer, mammal).
7 subclass(sardine, fish).
8 subclass(penguin, bird).
9 subclass(reindeer, herbivore).
```

(b) Subclasses definitions

Listing 7.1: T-Box definition

We must now create a rule to ensure that the subclass relation satisfies the transitive property:

$$X \text{ subclass of } Y \wedge Y \text{ subclass of } Z \Rightarrow X \text{ subclass of } Z.$$

In our ontology, for example, this rule is needed to establish that a penguin is an animal, but not a mammal. The most natural way, in a logic

programming language, to enforce the transitive property is through a recursive definition. Listing 7.2 shows how the rule is defined: lines 1 and 2 implement the base case of the recursion, while lines 4, 5, and 6 implement the recursive call.

```
1 subClass(C1, C2) :-
2     subclass(C1, C2).
3 subClass(C1, C2) :-
4     subclass(C1, C3),
5     subClass(C3, C2).
```

Listing 7.2: Recursive definition of subclass predicate

It is important to point out that the predicate we are defining in Listing 7.2 is different from the one used in Listing 7.1. The predicate `subclass(C1, C2)` (where `C1` and `C2` are classes) is used to construct the subclass hierarchy with a list of facts. The predicate `subClass(C1, C2)` captures the transitive property we are interested in. This one is the predicate that must be used in queries and in any subsequent rules whenever it is necessary to reason over the class hierarchy.

7.2.2 Individuals and their relationships

We define individuals by assigning them to a class; to do so, we use the predicate `isa(I, C)` where `I` is an individual, `C` is a class. Relationships, on the other hand, are defined through the predicate `haspropertyy(I1, R, I2)` that means individual `I1` is in the relation `R` with individual `I2`. Listing 7.3 is a collection of facts that populate our KB, and that define the relationships between individuals.

```
isa(beary, polar_bear).
isa(sealy, seal).
isa(reiny, reindeer).
isa(reiny_b, reindeer).
isa(reiny_c, reindeer).
isa(sardy, sardine).
isa(penguy, penguin).
```

(a) Individuals definitions

```
haspropertyy(beary, hunt, sealy).
haspropertyy(beary, hunt, penguy).
haspropertyy(sealy, hunt, penguy).
haspropertyy(penguy, hunt, sardy).
haspropertyy(reiny, hunt, sardy).
haspropertyy(reiny_b, sonOf, reiny).
haspropertyy(reiny_c, sonOf, reiny_b).
```

(b) Relationships definitions

Listing 7.3: T-Box definition

The predicate `isa(I, C)` allows us to define the class of an individual, but it does not take the class hierarchy into account. For example, we know that Beary is a polar bear, but we cannot infer that he is a mammal or an animal.

To also consider the superclasses of an individual's class, we define the predicate `isA(I, C)`. This predicate, thanks to the recursive definition of `subClass(C1, C2)`, permit to assign to an individual all the classes inherited from the class defined through the predicate `isa(I, C)`. Listing 7.4 shows the implementation of rule `isA(I, C)`.

```
isA(I, C) :-
    isa(I, C).
isA(I, C) :-
    isa(I, C2),
    subClass(C, C2).
```

Listing 7.4: isA predicate

As for the predicates `subclass(C1, C2)` and `subClass(C1, C2)`, also `isa(I, C)` must be used only in the definition of an individual's class, while `isA(I, C)` is to be used in queries and future rules.

7.2.3 Other rules

The KB that we have built so far contains classes, individuals, and relationships between individuals; we now want to add rules that allow us to extract new knowledge from the facts contained in the ontology.

In particular, we now present the rules to: assign a class to an individual, identify the ancestors of an individual, define 2 classes as disjoint, and find inconsistencies in the KB.

ASSIGNING A CLASS: The rules defined in Listing 7.5 allow us to assign the class carnivore and predator/prey to an individual. These rules are an example of how logical programming languages are easy to interpret: we can read Listing 7.5 as "If an animal hunts, it is a carnivore and it is a predator. If it is hunted, it is a prey."

```
isA(I, carnivore) :-
    hasproperty(I, hunt, _).
isA(I, predator) :-
    hasproperty(I, hunt, _).
isA(I, prey) :-
    hasproperty(_, hunt, I).
```

Listing 7.5: Class attribution rules

ANCESTORS: Similarly to what was done for the definition of `subClass(C1, C2)`, we can define the predicate `hasproperty(I1, ancestor, I2)`. Listing 7.6 shows the basic implementation.

```
hasproperty(I1, ancestor, I2) :-
    hasproperty(I2, sonOf, I1).
hasproperty(I1, ancestor, I2) :-
    hasproperty(I2, sonOf, I3),
    hasproperty(I1, ancestor, I3).
```

Listing 7.6: Ancestor definition

Although in an ontology that deals with the animal kingdom this predicate may seem of little interest, it provides us with a pretext to:

- define a predicate that can be expanded with other rules: in Listing 7.6 we define only the base step and the recursive call, but by imagining

building an ontology that deals with people, we can enrich the predicate with numerous rules that reflect family relationships between humans;

- have a predicate that can trigger many recursive calls[4]: the number of individuals populating an ontology can be considerable and can increase over time; moreover, if complex relationships are also considered —such as those suggested in the previous point— the number of ancestors of an individual can be very high and not easily known a priori.

DISJOINT CLASSES AND INCONSISTENCY: In our ontology, logical inconsistency is due to the fact that the reindeer Reiny is both herbivorous and carnivorous, but the two classes are disjoint. Let us therefore see how to express this disjointness in Prolog and how to capture the inconsistency in our KB.

Listing 7.7 shows how to define 2 disjoint classes and the consequences this has for individuals. At line 1 we see that to state that carnivores and herbivores are disjoint classes, a single fact is sufficient. The other lines show the consequences that disjointness has on individuals: if the classes `C` and `C2` are disjoint (line 4) and `I` belongs to class `C2` (line 5), then `I` does not belong to class `C` (line 3). `logicNot(P)` is a predicate that indicates that the proposition `P` is false[5].

```
1 disjointClasses(carnivore, herbivore).
2
3 logicNot(isA(I, C)) :-
4     disjointClasses(C, C2),
5     isA(I, C2).
```

Listing 7.7: Disjoint classes

Identifying the inconsistency of our KB means finding a statement that is both true and false. In particular, Reiny is carnivorous because she hunts (line 5 Listing 7.3), but she cannot be so because she is a reindeer and reindeers are herbivores (line 9 Listing 7.1).

It is possible to identify a proposition that is simultaneously true and false with the rule shown in Listing 7.8. The head of the rule is a list composed of a string (which serves only to print an informational message) and the proposition `P`. For the predicate to be true, the proposition `P` must be both false and true.

```
error(['A term cannot be true and false.', P]) :- logicNot(P), P.
```

Listing 7.8: Error rule

We can now test the consistency of the KB by asking the Prolog interpreter whether there exists a `P` that is simultaneously false and true. The result of the query is shown in the following listing:

4 It will be useful in Section 7.3.2.

5 Providing a detailed explanation of this predicate is beyond the scope of this work; in [37] the introduction of `logicNot(P)` is justified in order to bring the open world assumption into Prolog.

```
?- error(X).
    X = ['A term cannot be true and false.', isA(reiny, carnivore)].
```

7.2.4 Final result

Thanks to this translation process, we have obtained a KB expressed in Prolog equivalent to the original ontology expressed in OWL. Although with some simplifications, we were able to implement in Prolog the most important constructs offered by OWL, and we built a set of rules to perform inference on the newly created KB. The translation process can be further generalized to more advanced OWL constructs by following [37].

7.3 QA-PROLOG VERSION

The KB we have just obtained[6] is not compatible with the QA-Prolog pipeline. As presented in Section 6.1.3, QA-Prolog does not implement all the features of Prolog. In particular, lists and complex terms are not supported, and rules cannot be recursive.

In this section we will see our contribution on how to modify unsupported rules in order to make them compatible with the pipeline described in Chapter 6.

7.3.1 Lists and complex terms

In our KB we introduced a single list, in Listing 7.8, exclusively to print an additional message to explain the inconsistent proposition. The list can simply be removed, or we can use an additional variable that will be bound to the message we want to print.

Complex terms are predicates that have one or more predicates as arguments. The complex term present in our KB is `logicNot(isA(I, C))` introduced in Listing 7.7 line 3. We used this predicate to state that if an individual belongs to a class `C1`, disjoint from class `C2`, then the individual cannot belong to `C2`.

We can replace the complex term with a new predicate that represents the same concept: `isNotA(I, C)`. This new predicate forces us to modify also the rule for capturing inconsistencies in the KB. Listing 7.9 shows the final result.

Eliminating the predicate `logicNot(P)` where `P` is a proposition, therefore possibly a predicate, forces us to define a new predicate for each statement we want to negate, and to test each *predicate-predicate_negation* pair to check for possible inconsistencies.

In our KB, only class membership can generate inconsistency, so a single rule is sufficient. In general, other definitions of the predicate `error` may be necessary to account for all possible causes of inconsistency[7].

6 Available at https://github.com/DavideCamino/Thesis/blob/main/Code/2_QA-prolog/3_QA-prolog/KB1/animal.pl.

7 However, this also gives us the possibility to define more precise error messages.

```
isNotA(I, C) :-
    disjointClasses(C, C2),
    isA(I, C2).

error(I, C, M) :-
    isA(I, C),
    isNotA(I, C),
    M = 'Individual X cannot belong and not belong to class Y'.
```

Listing 7.9: List and complex term removal

7.3.2 Unfolding

In our KB there are two rules that are defined recursively: `subClass(C1, C2)` and `hasproperty(I1, R, I2)`.

To eliminate recursion, we can *unfold* the recursive calls. The procedure is analogous to the *loop unrolling* performed by modern compilers to speed up the execution of machine code. Loop unrolling consists of transforming a loop of instructions into a repeated sequence of instructions; this increases the length of the code but reduces the number of loop condition checks.

Unfolding consists of developing the recursion by explicitly repeating the function call; in this way the recursive call is replaced by many calls to the function `A` in the function `B`.

SUBCLASSES: To eliminate recursion in the rule `subClass(C1, C2)` we can remove the rule with the recursive call, replacing it with a sufficient number of other rules to cover the entire class hierarchy; each new rule adds one call and allows moving one level higher in the hierarchy.

Our ontology has a maximum depth of 3, therefore at most two recursive calls are needed to start from any node in the class hierarchy and reach the root. Listing 7.10 shows the updated version of the rule `subClass(C1, C2)` (to clarify the procedure, unfolding is developed up to three calls, even though only two were necessary to cover the entire class hierarchy).

```
subClass(C1, C2) :-
    subclass(C1, C2).
subClass(C1, C3) :-
    subclass(C1, C2),
    subclass(C2, C3).
subClass(C1, C4) :-
    subclass(C1, C2),
    subclass(C2, C3),
    subclass(C3, C4).
```

Listing 7.10: Subclasses unfolding

ANCESTOR: The issue with the predicate `hasproperty(I1, R, I2)` concerns the ancestor-descendant relationship. Indeed, its definition was very similar to the rule `subClass(C1, C2)`. In this case, however, recursion is also present in the base step, as can be seen in lines 1 and 2 of Listing 7.6. Listing 7.11 shows a possible solution. About this code it is important to note:

- unlike Listing 7.10, we define many new predicates: this is useful if there were multiple ways to define new ancestors starting from those already identified[8];
- the predicate `hasProperty(I1, R, I2)` has been added: this must be used in queries, while `hasproperty(I1, R, I2)` is used only to define relationships between individuals.
- we stopped, arbitrarily, after 10 ancestors; in general there is no way to know how many recursive calls should be unfolded to be sure of capturing all the ancestors of an individual.

```
ancestor1(I1, I2) :-
    hasproperty(I2, sonOf, I1).
ancestor2(I1, I2) :-
    ancestor1(I1, I3),
    hasproperty(I2, sonOf, I3).
ancestor3(I1, I2) :-
    ancestor2(I1, I3),
    hasproperty(I2, sonOf, I3).
    ...
ancestor9(I1, I2) :-
    ancestor8(I1, I3),
    hasproperty(I2, sonOf, I3).

ancestor_unfold(I1, I2) :-
    ancestor1(I1, I2);
    ancestor2(I1, I2);
    ...
    ancestor9(I1, I2).

hasProperty(I1, P, I2) :-
    hasproperty(I1, P, I2).
hasProperty(I1, ancestor,I2) :-
    ancestor_unfold(I1, I2).
```

Listing 7.11: Ancestor unfolding

7.3.3 Stopping criterion

As mentioned above, it is not always possible to know in advance how many recursive calls will be performed to capture all the aspects we are interested in. In general, we can distinguish two cases: T-Box and A-Box.

T-BOX: The T-Box describes our domain of interest and should be stable. Once a description of the world suitable for our purpose has been developed, it is rare for it to change; in our case, a modification of the T-Box could occur only if new species were discovered, which is not frequent.

If the rules with recursive calls involve the T-Box, it can be possible to know the maximum number of recursive calls needed to capture all relevant

8 Listing .2 in the appendix shows an example for clarification.

information and perform sufficient unfolding to allow an adequate number of function calls.

An example of this stopping criterion was shown in the rewriting of the rule `subClass(C1, C2)`. Once the class hierarchy is known, it is possible to compute the maximum depth and use it as a criterion to stop the unfolding process.

A-BOX: The A-Box contains the individuals populating the ontology. For this reason, the A-Box is much more subject to change than the T-Box and, as discussed when presenting the ancestor-descendant relationship, relationships between individuals can also be very complex.

Each time an individual is added to the KB or its relationships are modified, the depth of the recursive calls may change.

With regard to the A-Box, the stopping criterion for unfolding must be decided in advance or estimated through heuristics where possible. It is not possible to know in advance the maximum number of recursive calls without performing reasoning, but this would be counterproductive since the objective of the translation is to perform reasoning on a quantum annealer.

7.3.4 Equivalence between the KBs

Before replacing the KB with unsupported construct and recursive rules with the modified one, we must ensure that there is equivalence between the two KBs.

To show equivalence between recursive calls and the unfolded version, it would be possible to construct a proof based on the principle of induction, possibly with the aid of a proof assistant.

Such a constructive proof is not the objective of this work. To convince ourselves of the equivalence and to take into account also other modifications, we test the behavior of queries that use rewritten rules to ensure their correctness.

As far as possible, the unfolding process was carried out so as to make it transparent to the end user. The result was achieved with regard to queries related to the subclass hierarchy and the retrieval of an individual's ancestors.

As for error detection, however, the various rules that define the predicate `error` may have different numbers of parameters. For each number of parameters it will be necessary to test the predicate[9] in order to ensure that all inconsistencies are identified.

Listing 7.12 shows the result of the queries that use the modified rules to make them compatible with the QA-Prolog pipeline. The result of the queries is as expected: we obtained all and only the results that we would also have obtained by querying the original KB. This intuitively convinces us of the equivalence of the two KBs.

9 Not for each definition of the predicate; for example, if there are two definitions with 3 parameters and one with 4, it will be necessary to test `error` twice.

```
?- subClass(reindeer, X).
    X = mammal ;
    X = herbivore ;
    X = animal ;
    false.
```

(a) Subclasses

```
?- hasProperty(X, ancestor, reiny_c).
    X = reiny_b ;
    X = reiny ;
    false.
```

(b) Ancestor

```
?- error(X, Y, Z).
    X = reiny,
    Y = carnivore,
    Z = 'Individual X cannot belong and not belong to class Y'
```

(c) Error

Listing 7.12: Query results

7.4 FINAL RESULT

The obtained KB[10] is compatible with the QA-Prolog pipeline. In this section we go through the entire transformation pipeline until obtaining the Hamiltonian that represents the KB together with the associated query, and we discuss the results obtained.

Formulating a query is essential because it defines the outputs of our program and, therefore, allows us to build the digital circuit, which is the first step of the pipeline.

To evaluate the result of the translation, we do not consider the outcome of the query evaluation because, as already shown in the chapter on the pipeline in Section 6.2.3, the simulators offered by the Ocean framework are able to find the solution only for very small problem instances. What we consider instead is the dimension of the minimization problem obtained at the end of the pipeline, extracting the number of qubits composing the polynomial to be minimized, and attempting to provide a justification for the different dimensions we obtain.

7.4.1 Query

We construct different queries that test different facts and rules defined in our KB. We consider both simple queries, whose answer can be found simply by scanning the facts explicitly listed in the KB, and increasingly complex queries that require applying one or more inference rules.

The queries we test are:

1. `?- class(animal).`
2. `?- class(X).`
3. `?- subClass(seal, X).`
4. `?- isA(sardy, X).`

10 Available at https://github.com/DavideCamino/Thesis/blob/main/Code/2_QA-prolog/3_QA-prolog/KB1/animal_unfold.pl.

5. `?- error(X, Y, Z).`
6. `?- hasProperty(I1, ancestor, reiny_c).`

In the results we will refer to the queries directly by the number associated with them in this list.

7.4.2 Results

As can be seen from Figure 7.3, the number of *qubits* required to represent the problem varies significantly, from a few dozen, for the first queries to almost ten thousand *qubits*, for the query related to the ancestors of an individual.

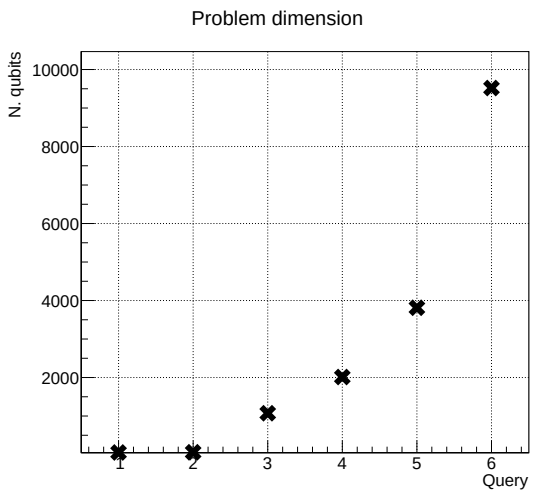

Figure 7.3: Problem size

The large difference in the number of *qubits* used is due to the complexity of the query, to how many facts and rules must be used in order to find its solution.

For simple queries such as the first two, it is sufficient to scan the list of facts. Query 3, which generates a problem with 1072 Boolean variables, requires the construction of the entire class hierarchy. Query 4 (2013 *qubits*) also requires the class hierarchy; moreover, all the rules that allow us to define an individual as belonging to a certain class must be considered (rules of Listing 7.5).

Query 5 adds, to all the rules necessary to solve query 4, also the disjointness between classes and the rule that defines inconsistencies. The number of variables thus increases to 3812. Finally, query 6 uses relationships between individuals and in particular requires the relation that defines ancestors, which was unfolded in Listing 7.11. Precisely because of the unfolding, the number of rules increases significantly and, consequently, the dimension of the problem grows up to 9519 *qubits*.

From the obtained results, it is clear that, depending on the query, we obtain Hamiltonians of very different sizes. This is advantageous because it allows us not to consider the entire KB for each query, but to extract only the part necessary to answer our specific question.

One may then ask which step of the pipeline is responsible for this extraction. By observing the files in the QA-Prolog working directory, it can be noted that the Verilog file is the same for all queries, whereas the EDIF file, synthesized by Yosys (Section 6.2.2), is radically different from one query to another.

The optimization therefore occurs at the level of the logic circuit: once the output of our circuit has been defined through the query, Yosys is able to discard all those parts of the circuit described in Verilog that are not necessary to define the output. Yosys generates a EDIF file using only the Verilog

module connected, in circuit term, to the output. This step, as highlighted by the graph in Figure 7.3, is able to significantly reduce the dimension of the problem.

7.5 CONCLUSION

In this chapter we have shown how it is possible to move from an ontology expressed in OWL to a KB expressed in Prolog on which it is possible to highlight inconsistencies in the facts and perform reasoning as on the original ontology.

We have recalled some of the limitations of QA-Prolog, showing how they impacted our translation; we proposed solutions that led us to rewrite some rules, and, above all, to unfold recursive calls.

Finally, we showed that the obtained KB is effectively compatible with the QA-Prolog pipeline by going through all the steps until obtaining a Hamiltonian, whose dimensions we discussed by relating them to the query and consequently to the facts and rules necessary to answer that query.

At a high level, this chapter can be seen as an additional upstream step of the pipeline: QA-Prolog, through successive rewritings into different formats that logically always represent the same problem with the same solution, starts from a KB in Prolog and reaches a representation suitable for a quantum annealer. In this chapter we added the rewriting from OWL to Prolog, increasing by one step the chain of rewritings.

BIBLIOGRAPHY

[1] Barry Smith. "Ontology". In: (2012).

[2] Gian Piero Zarri. "Ontologies and Their Practical Implementation". In: *Encyclopedia of Database Technologies and Applications*. IGI Global Scientific Publishing, 2005, pp. 438–449.

[3] Thomas R Gruber. "A translation approach to portable ontology specifications". In: *Knowledge acquisition* 5.2 (1993), pp. 199–220.

[4] Marco Fossati, Emilio Dorigatti, and Claudio Giuliano. "N-ary relation extraction for simultaneous T-Box and A-Box knowledge base augmentation". In: *Semantic Web* 9.4 (2018), pp. 413–439.

[5] Giuseppe De Giacomo, Maurizio Lenzerini, et al. "TBox and ABox reasoning in expressive description logics." In: *KR* 96.316-327 (1996), p. 10.

[6] Markus Krötzsch, Frantisek Simancik, and Ian Horrocks. "A description logic primer". In: *arXiv preprint arXiv:1201.4089* (2012).

[7] Pascal Hitzler et al. *OWL 2 Web Ontology Language Primer (Second Edition)*. W3C Recommendation REC-owl2-primer-20121211. World Wide Web Consortium (W3C), Dec. 2012. URL: https://www.w3.org/TR/2012/REC-owl2-primer-20121211/.

[8] Pascal Hitzler. "A review of the semantic web field". In: *Communications of the ACM* 64.2 (2021), pp. 76–83.

[9] Stefano Borgo et al. "DOLCE: A descriptive ontology for linguistic and cognitive engineering". In: *Applied ontology* 17.1 (2022), pp. 45–69.

[10] Boris Motik, Peter F. Patel-Schneider, and Bernardo Cuenca Grau. *OWL 2 Web Ontology Language: Direct Semantics (Second Edition)*. W3C Recommendation REC-owl2-direct-semantics-20121211. World Wide Web Consortium (W3C), Dec. 2012. URL: https://www.w3.org/TR/2012/REC-owl2-direct-semantics-20121211/.

[11] Ilianna Kollia, Birte Glimm, and Ian Horrocks. "Query answering over SROIQ knowledge bases with SPARQL". In: *Proceedings of the 24th International Workshop on Description Logics, Barcelona, Spain*. 2011, pp. 13–16.

[12] Franz Baader. *The description logic handbook: Theory, implementation and applications*. Cambridge university press, 2003.

[13] Birte Glimm et al. "HermiT: an OWL 2 reasoner". In: *Journal of automated reasoning* 53.3 (2014), pp. 245–269. URL: http://www.hermit-reasoner.com/.

[14] Scott Pakin. "Performing fully parallel constraint logic programming on a quantum annealer". In: *Theory and Practice of Logic Programming* 18.5-6 (2018), pp. 928–949.

[15] William M Kaminsky, Seth Lloyd, and Terry P Orlando. "Scalable superconducting architecture for adiabatic quantum computation". In: *arXiv preprint quant-ph/0403090* (2004).

[16] Los Alamos National Laboratory / Scott Pakin. *QA-Prolog: Quantum Annealing Prolog*. Accessed: 2026-02-13, GitHub repository. URL: https://github.com/lanl/QA-Prolog.

[17] Umer Farooq, Zied Marrakchi, and Habib Mehrez. "FPGA architectures: An overview". In: *Tree-Based Heterogeneous FPGA Architectures: Application Specific Exploration and Optimization* (2012), pp. 7–48.

[18] William F Clocksin and Christopher S Mellish. *Programming in PROLOG*. Springer Science & Business Media, 2003.

[19] Patrick Blackburn, Johan Bos, and Kristina Striegnitz. *Learn Prolog Now! – Free Online Version*. Accessed: 2026-02-13. 2012. URL: https://lpn.swi-prolog.org/lpnpage.php?pageid=online.

[20] Robert Kowalski and Steve Smoliar. "Logic for problem solving". In: *ACM SIGSOFT Software Engineering Notes* 7.2 (1982), pp. 61–62.

[21] Christopher John Hogger. *Introduction to logic programming*. Academic Press Professional, Inc., 1984.

[22] SWI-Prolog Developers. *SWI-Prolog Reference Manual*. Accessed: 2026-02-20. SWI-Prolog. URL: https://www.swi-prolog.org/pldoc/doc_for?object=manual.

[23] Jan Wielemaker et al. "SWI-Prolog". In: *Theory and Practice of Logic Programming* 12.1-2 (2012), pp. 67–96. ISSN: 1471-0684.

[24] Markus Triska. "The finite domain constraint solver of SWI-Prolog". In: *International Symposium on Functional and Logic Programming*. Springer. 2012, pp. 307–316.

[25] Markus Triska. *clpfd: Constraint Logic Programming over Finite Domains*. https://github.com/triska/clpfd. GitHub repository, accessed 2026-02-20.

[26] Markus Triska. *clpz: Constraint Logic Programming over Integers*. https://github.com/triska/clpz. GitHub repository, accessed 2026-02-20.

[27] Roshan P James, Gerardo Ortiz, and Amr Sabry. "Quantum computing over finite fields". In: *arXiv preprint arXiv:1101.3764* (2011).

[28] Mohamed W Hassan, Scott Pakin, and Wu-chun Feng. "C to D-wave: a high-level C compilation framework for quantum Annealers". In: *2019 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE. 2019, pp. 1–8.

[29] Scott Pakin. "A quantum macro assembler". In: *2016 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE. 2016, pp. 1–8.

[30] Michael R Garey and David S Johnson. *Computers and intractability*. Vol. 29. wh freeman New York, 2002.

[31] Mike Gordon. "The semantic challenge of Verilog HDL". In: *Proceedings of tenth annual IEEE symposium on logic in computer science*. IEEE. 1995, pp. 136–145.

[32] Scott Pakin. "Targeting classical code to a quantum annealer". In: *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. 2019, pp. 529–543.

[33] Clifford Wolf, Johann Glaser, and Johannes Kepler. "Yosys-a free Verilog synthesis suite". In: *Proceedings of the 21st Austrian Workshop on Microelectronics (Austrochip)*. Vol. 97. 2013.

[34] Robert Brayton and Alan Mishchenko. "ABC: An academic industrial-strength verification tool". In: *International Conference on Computer Aided Verification*. Springer. 2010, pp. 24–40.

[35] Hongyu Zhang, Yuan-Fang Li, and Hee Beng Kuan Tan. "Measuring design complexity of semantic web ontologies". In: *Journal of Systems and Software* 83.5 (2010), pp. 803–814.

[36] Mark A Musen. "The protégé project: a look back and a look forward". In: *AI matters* 1.4 (2015), pp. 4–12.

[37] Ken Samuel et al. "Translating owl and semantic web rules into prolog: Moving toward description logic programs". In: *Theory and Practice of Logic Programming* 8.3 (2008), pp. 301–322.

APPENDIX 1

```
// Verilog version of Prolog program friends.pl
// Conversion by QA-Prolog, written by Scott Pakin <pakin@lanl.gov>
//
// This program is intended to be passed to edif2qmasm, then to qmasm, and
// finally run on a quantum annealer.
//
// Note: This program uses 3 bit(s) for atoms and 1 bit(s) for (unsigned)
// integers.

// Define all of the symbols used in this program.
`define alice   3'd0
`define bob     3'd1
`define charlie 3'd2
`define enemies 3'd3
`define friends 3'd4
`define hates   3'd5

// Define hates(atom, atom).
module \hates/2 (A, B, Valid);
  input [2:0] A;
  input [2:0] B;
  output Valid;
  wire [1:0] $v1;
  assign $v1[0] = A == `alice;
  assign $v1[1] = B == `bob;
  wire [1:0] $v2;
  assign $v2[0] = A == `bob;
  assign $v2[1] = B == `charlie;
  assign Valid = &$v1 | &$v2;
endmodule

// Define enemies(atom, atom).
module \enemies/2 (A, B, Valid);
  input [2:0] A;
  input [2:0] B;
  output Valid;
  wire $v1;
  \hates/2 \hates_pujkx/2 (A, B, $v1);
  wire $v2;
  \hates/2 \hates_DoUbH/2 (B, A, $v2);
  assign Valid = &$v1 | &$v2;
endmodule

// Define friends(atom, atom).
module \friends/2 (A, B, Valid);
  input [2:0] A;
  input [2:0] B;
```

```
  output Valid;
  (* keep *) wire [2:0] C;
  wire [2:0] $v1;
  \enemies/2 \enemies_GcbiL/2 (A, C, $v1[0]);
  \enemies/2 \enemies_GNLnA/2 (C, B, $v1[1]);
  assign $v1[2] = A != B;
  assign Valid = &$v1;
endmodule

// Define Query(atom, atom).
module Query (P1, P2, Valid);
  input [2:0] P1;
  input [2:0] P2;
  output Valid;
  wire $v1;
  \friends/2 \friends_KMGjP/2 (P1, P2, $v1);
  assign Valid = &$v1;
endmodule
```

Listing .1: QA-Prolog compilation result

```
ancestor1(I1, I2) :-
    hasproperty(I2, sonOf, I1).
ancestor1(I1, I2) :-
    hasproperty(I2, sonOf, I3).
    hasproperty(I1, marry, I3).
ancestor2(I1, I2) :-
  hasproperty(I1, grandParentOf, I2).
ancestor2(I1, I2) :-
    ancestor1(I1, I3),
    hasproperty(I2, sonOf, I3).
ancestor2(I1, I2) :-
    ancestor1(I3, I2),
    hasproperty(I3, sonOf, I4).
    hasproperty(I1, marry, I4).
ancestor3(I1, I2) :-
  ancestor1(I3, I2)
  hasproperty(I1, grandParentOf, I2).
ancestor3(I1, I2) :-
  ancestor2(I3, I2)
  hasproperty(I3, sonOf, I1).
ancestor3(I1, I2) :-
  ancestor2(I3, I2)
  hasproperty(I3, sonOf, I4).
  hasproperty(I1, marry, I4).
  ...
```

Listing .2: Ancestor definition extended to family relation